

Классын удамшил ба түүний хэрэглээ , ач холбогдол (Лабораторийн ажил №7.)

Багийн нэр : IRIS

Багийн ахлагч : О.Эрдэнэбулган

Багийн гишүүд :

- МУИС, МТЭС , 24B1NUM1959@stud.num.edu.mn , О.Эрдэнэбулган
- МУИС, МТЭС , 24B1NUM2023@stud.num.edu.mn , Д.Сундуйжав
- МУИС, МТЭС , 23B1NUM0135@stud.num.edu.mn , Б.Тэмүүлэл

Багийн зорилго :

Энэхүү лабораторийн ажлаар бид C++ хэлний удамшил түүнтэй холбоотой ойлголтуудыг онолын түвшинд судалж классууд-ын удамшлыг өгөгдсөн бодлого дээр хэрэгжүүлэх зорилготой. Мөн багаар ажиллах чадварыг сайжруулах, ажлын гүйцэтгэл, хурд, зохион байгуулалтын чадварыг суралцахаар зорьсон.

Багийн гишүүн бүрийн оролцоо:

- Сундуйжав- Дутуу хийсэн класс дээр нэмэлтээр зааврын дагуу тооцоолол хийх байгуулагч функцүүд байгуулж объектуудыг байгуулсан бөгөөд тайлан бичилтэд хамтарч ажиллав.
- Тэмүүлэл- Лабораторийн тайлан бичихэд багийн гишүүдтэйгээ хамтарч ажиллав.

Багаар ажиллахад тулгарсан асуудлууд:

- Уулзалтын цаг төлөвлөх
- Ажлын хуваарьлалт
- Олон хувилбартай даалгавруудаас сонгох

Багаар ажиллаад сурсан зүйл:

Багаар ажиллах явцад би хамтын ажиллагааны ач холбогдлыг ойлгож, бусадтай харилцах, санал солилцох чадвараа хөгжүүлсэн. Бие биенийхээ санааг сонсож, нэг зорилгод хүрэхийн тулд үүрэг хариуцлагаа зөв хуваарилах нь ажлыг илүү үр дүнтэй болгодгийг мэдэрсэн.

Мөн асуудал гарсан үед ганцаараа шийдэхээс илүү багаар хэлэлцэж шийдэх нь илүү оновчтой шийдэлд хүргэдэг гэдгийг ойлгосон. Багаар ажиллах нь зөвхөн ажлыг хийхээс гадна харилцааны ур чадвар, тэвчээр, хариуцлагыг хөгжүүлдэг чухал туршлага байлаа.

1. ОРШИЛ

Объект хандалтат программчлалын үндсэн ойлголтод ерөнхийлөл, нарийвчлал, удамшил зэрэг чухал зарчмууд багтдаг. Эдгээр нь програм хангамжийг бодит ертөнцийн объектуудтай төстэй байдлаар загварчлах боломж олгож, кодыг илүү ойлгомжтой, дахин ашиглах боломжтой, цэгцтэй болгодог. Ингэснээр системийг цаашид өргөтгөх болон засварлахад илүү хялбар болдог.

Энэхүү тайлан нь дээрх ойлголтуудыг онолын түвшинд тайлбарлахаас гадна програм хангамж боловсруулахад хэрхэн практик дээр ашиглагддагийг товч бөгөөд ойлгомжтой байдлаар харуулах зорилготой.

2. ЗОРИЛГО

Энэхүү тайлангийн зорилго нь объект хандалтат программчлалын ерөнхийлөл, нарийвчлал болон удамшилын үндсэн ойлголтуудыг судалж ойлгох явдал юм. Мөн эдгээр ойлголтууд бодит програм дээр хэрхэн хэрэгждэгийг жишээгээр тайлбарлаж , практик хэрэглээг нь ойлгохыг зорьсон.

3. ОНОЛЫН СУДАЛГАА

3.1 Удамшил ба байгуулагч , устгагч функц

Удамшил гэдэг нь эх классаас түүний гишүүн өгөгдөл болон функцийг өвлөн авч удамшуулахыг хэлдэг.

Байгуулагч гэдэг нь классын объект үүсэх үед дуудагдаж анхны утгыг оноодог тусгай байгуулагч юм. Мөн удамшсан классын объект үүсэх үед эх классын байгуулагч түрүүлж дуудагдан дараа нь охин классын байгуулагч дуудагдана.

```
#include<bits/stdc++.h>
using namespace std ;
```

```

// эх класс
class Person {
    public :
        // эх классын анхдагч байгуулагч
        Person(){
            cout << "A baiguulagch duudagdlaa" << endl ;
        }
        // эх классын устгагч
        ~Person(){}
} ;

class Teacher : public Person {
    public :
        // охин классын анхдагч байгуулагч
        Teacher(){
            cout << "B baiguulagch duudagdlaa" << endl ;
        }
        // охин классын устгагч
        ~Teacher(){}
} ;

int main(){
    // удамшсан классын объект
    Teacher t1 ;

    // объект үүсэх үед дараах дарааллаар дуудагдана.
    // А байгуулагч
    // В байгуулагч

    // объект устгах үед дараах дарааллаар дуудагдана.
    // В устгагч
    // А устгагч

}

```

3.2 Ерөнхийлөл

Ерөнхийлөл гэдэг нь удамшсан классуудад байх нийтлэг шинжийг эх класст үүсгэн зарлаж хэрэглэхийг хэлнэ. Үүнийг ашигласнаар кодыг дахин давтах шаардлагагүй болно. C++ хэл дээр үүнийг удамшил ашиглан хэрэгжүүлдэг.

```
class Animal {
public:
    void eat(){
        cout << "Animal is eating." << endl ;
    } ;

class Dog : public Animal {
    void bark(){
        cout << "Dog is barking." << endl ;
    } ;

int main(){
    // объект үүсгэх
    Dog d ;
    // удамшсан функц
    d.eat() ;
    // өөрийн функц
    d.bark() ;
}
```

3.3 Нарийвчлал

Нарийвчлал гэдэг нь эх класс дээр үндэслэн илүү тодорхой , онцлог шинж бүхий хүүхэд класс үүсгэх үйл явц юм. Удамшсан класс нь эцэг классын шинж , функцуудыг удамшиж авч , шаардлагатай үед шинэ функц нэмэх эсвэл override хийж өөрчилдөг.

Энэ нь бодит ертөнцийн объектуудыг илүү нарийвчлалтай загварчлах боломж олгодог бөгөөд C++ хэл дээр удамшлаар хэрэгждэг.

```

// эх класс
class Animal {
    public :
        virtual void eat(){
            cout << "Animal is eating." << endl ;
        }

        void sleep(){
            cout << "it is sleeping." << endl ;
        } ;
} ;

//охин класс
class Dog : public Animal {
    public :
        void eat(){
            cout << "dog is eating." << endl ;
        }

        void bark(){
            cout << "dog is barking" << endl ;
        }
} ;

int main(){
    Dog d ;
    // override хийгдсэн
    d.eat() ;
    // эцгээс удамшсан
    d.sleep() ;
    // өөрийн функц
    d.bark() ;
}

```

3.4 Функц дахин программчлах

Функц дахин программчлах гэдэг нь удамшсан класс болон эцэг классын ижил нэртэй, ижил параметртэй функцийг үр дүнг өөр байхаар программчлахыг хэлнэ.

```
class Person {
    public :
        virtual void learn(){
            cout << "Person is learning." << endl ;
        }
} ;

class Teacher : public Person {
    public :
        void learn(){
            cout << "Teacher is learning." << endl ;
        }
} ;

int main(){
    Person* p ;
    Person p1 ;
    Teacher t ;
    p = &t ;
    // Teacher is learning.
    p->learn() ;
    // P erson is learning.
    p1.learn() ;
}
```

4. ХЭРЭГЖҮҮЛЭЛТ

```
#include<bits/stdc++.h>

#include<math.h>

using namespace std ;

struct Point{
    float x;
    float y;
} typedef Point ;

class Shape{
    protected :
        Point points[100];
    public :
        Shape(){}
        // устгагчийг удамшуулж ажиллуулах
        virtual ~Shape(){}
} ;

class Shape2D : public Shape{
    protected :
        string name;
    public :
        // удамших байгуулагч
        Shape2D(string n) : Shape() {
            name = n;
```

```

    }

    // override хийж удамшуулна
    virtual float talbai() = 0 ;
    virtual float perimetr() = 0 ;
    string getName(){
        return name ;
    }

    // устгагчийг удамшуулж ажиллуулах
    virtual ~Shape2D(){}
} ;

class Circle : public Shape2D{
    protected :
        Point p1 ;
        float radius ;
    public :
        // удамших байгуулагч
        Circle(Point p1 , float radius , string name) :
        Shape2D(name) {
            this->p1 = p1 ;
            this->radius = radius ;
        }
        // override удамшина.
        float talbai() override {
            return 3.14 * radius * radius ;
        }
        // override удамшина.

```



```

        float perimetr() override {
            return 2 * 3.14 * radius ;
        }

        // устгагч удамшиж дуудагдана
        ~Circle(){}
    } ;

class Square : public Shape2D{
    protected :
        Point p1 , p2 , p3 , p4 ;
        float tal ;
    public :
        // удамших байгуулагч
        Square(Point topLeft, float length, string n) :
        Shape2D(n) {
            p1 = topLeft;
            p2 = {topLeft.x + length, topLeft.y};
            p3 = {topLeft.x + length, topLeft.y - length};
            p4 = {topLeft.x, topLeft.y - length};
            tal = length;
        }
        // override удамшина.
        float talbai() override {
            return tal * tal ;
        }
        // override удамшина.
        float perimetr() override {

```

```

        return 4 * tal ;
    }

    // устгагч удамшиж дуудагдана
    ~Square() {}
};

class Triangle : public Shape2D {
protected:
    Point p1, p2, p3;
    float tal1, tal2, tal3;
public:
    // удамших байгуулагч
    Triangle(Point top, float a, string n) : Shape2D(n) {
        p1 = top;
        float height = sqrt(3)/2 * a;
        p2 = {top.x - a/2, top.y - height};
        p3 = {top.x + a/2, top.y - height};

        tal1 = tal2 = tal3 = a;
    }
    // override удамшина.
    float talbai() override {
        return (sqrt(3)/4) * tal1 * tal1;
    }
    // override удамшина.
    float perimetr() override {

```

```

        return tal1 + tal2 + tal3;
    }

    // устгагч удамшиж дуудагдана
    ~Triangle(){}
};

// талбайгаар эрэмбэлэлт хийх
void bubbleSort(Shape2D** shapes, int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(shapes[j]->talbai() > shapes[j + 1]-
>talbai()) {
                swap(shapes[j], shapes[j + 1]);
            }
        }
    }
}

int main() {
    int n ;
    cin >> n ;

    // объектын хаягийг заах хувьсагч
    Shape2D** shapes = new Shape2D*[n];

    // санах ой нөөцлөн объект үүсгэх
    shapes[0] = new Circle({0,0}, 5, "Circle1");
    shapes[1] = new Square({0,10}, 4, "Square1");
    shapes[2] = new Triangle({0,10}, 6, "Triangle1");
    shapes[3] = new Circle({1,1}, 3, "Circle2");
    shapes[4] = new Square({2,8}, 2, "Square2");

```

```

shapes[5] = new Triangle({2,5}, 4, "Triangle2");

for(int i = 0; i < n - 1; i++){
    for(int j = 0; j < n - i - 1; j++){
        if(shapes[j]->talbai() > shapes[j + 1]->talbai()){
            swap(shapes[j], shapes[j + 1]);
        }
    }
}

cout << "=== Sorted by area ===\n";
// эрэмбэлэлт
for(int i = 0; i < n; i++){
    cout << shapes[i]->getName() << endl;
    cout << "Talbai: " << shapes[i]->talbai() << endl;
    cout << "Perimetr: " << shapes[i]->perimetr() << endl;
    cout << "-----" << endl;
}
// санах ойгоо чөлөөлөх
for(int i = 0; i < n; i++){
    delete shapes[i];
}

delete[] shapes;

```

}

5. ДҮГНЭЛТ

Энэхүү лабораторийн ажлаар Circle, Square, Triangle зэрэг классуудыг ашиглан олон объект үүсгэж, тэдгээрийг талбайн хэмжээгээр нь эрэмбэлэх кодыг хэрэгжүүллээ. Мөн эх класст байгуулагч функц тодорхойлж, удамшсан классууд нь эх классын параметертэй байгуулагчыг constructor chaining ашиглан дуудаж ажиллуулах зарчмыг ойлгож, практикт хэрэгжүүлсэн.

Энэ аргачлалын давуу тал нь кодын давтагдлыг багасгаж, нийтлэг шинж чанаруудыг нэг суурь класс дээр төвлөрүүлснээр системийн бүтэц илүү цэгцтэй, ойлгомжтой болдогт оршино. Мөн полиморфизм ашигласнаар бүх дүрсийг нэг төрлийн pointer-оор удирдаж, нэг алгоритмаар (талбайгаар эрэмбэлэх гэх мэт) боловсруулах боломж бүрддэг.

Харин сул тал нь динамик санах ойн удирдлага шаарддаг тул new/delete-ийг зөв ашиглахгүй бол memory leak үүсэх эрсдэлтэй. Мөн pointer ашиглалт ихэссэнээр кодын алдаа гарах магадлал нэмэгдэж, ойлгоход харьцангуй төвөгтэй болдог.

6. АШИГЛАСАН МАТЕРИАЛ

1. Объект хандлагат технологийн C++ програмчлал, Ж.Пүрэв, 2008, Улаанбаатар.

2. Inheritance , <https://cplusplus.com>

7. ХАВСРАЛТ

```
#include<bits/stdc++.h>

#include<math.h>

using namespace std ;


struct Point{

    float x;

    float y;

} typedef Point ;


class Shape{

    protected :

        Point points[100];

    public :

        Shape() {

        }

        virtual ~Shape() {

        }

} ;


class Shape2D : public Shape{

    protected :
```

```

        string name;
public :
    Shape2D(string n) : Shape() {
        name = n;
    }
    virtual float talbai() = 0 ;
    virtual float perimetr() = 0 ;
    string getName(){
        return name ;
    }
    virtual ~Shape2D(){}
} ;

class Circle : public Shape2D{
protected :
    Point p1 ;
    float radius ;
public :
    Circle(Point p1 , float radius , string name) :
Shape2D(name) {
        this->p1 = p1 ;
        this->radius = radius ;
    }
    float talbai() override {
        return 3.14 * radius * radius ;
    }
    float perimetr() override {

```

```

        return 2 * 3.14 * radius ;
    }
    ~Circle(){}
} ;

class Square : public Shape2D{
    protected :
        Point p1 , p2 , p3 , p4 ;
        float tal ;
    public :
        Square(Point topLeft, float length, string n) :
        Shape2D(n) {
            p1 = topLeft;
            p2 = {topLeft.x + length, topLeft.y};
            p3 = {topLeft.x + length, topLeft.y - length};
            p4 = {topLeft.x, topLeft.y - length};
            tal = length;
        }
        float talbai() override {
            return tal * tal ;
        }
        float perimetr() override {
            return 4 * tal ;
        }
        ~Square(){}
};

```



```

class Triangle : public Shape2D {
protected:
    Point p1, p2, p3;
    float tal1, tal2, tal3;
public:
    Triangle(Point top, float a, string n) : Shape2D(n) {
        p1 = top;
        float height = sqrt(3)/2 * a;
        p2 = {top.x - a/2, top.y - height};
        p3 = {top.x + a/2, top.y - height};

        tal1 = tal2 = tal3 = a;
    }
    float talbai() override {
        return (sqrt(3)/4) * tal1 * tal1;
    }
    float perimetr() override {
        return tal1 + tal2 + tal3;
    }
    ~Triangle(){}
};

```

```

void bubbleSort(Shape2D** shapes, int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(shapes[j]->talbai() > shapes[j + 1]-
>talbai()) {

```

```

        swap(shapes[j], shapes[j + 1]);
    }
}

}

int main() {
    int n = 6 ;

    Shape2D** shapes = new Shape2D*[n];

    shapes[0] = new Circle({0,0}, 5, "Circle1");
    shapes[1] = new Square({0,10}, 4, "Square1");
    shapes[2] = new Triangle({0,10}, 6, "Triangle1");
    shapes[3] = new Circle({1,1}, 3, "Circle2");
    shapes[4] = new Square({2,8}, 2, "Square2");
    shapes[5] = new Triangle({2,5}, 4, "Triangle2");

    for(int i = 0; i < n - 1; i++){
        for(int j = 0; j < n - i - 1; j++){
            if(shapes[j]->talbai() > shapes[j + 1]->talbai()){
                swap(shapes[j], shapes[j + 1]);
            }
        }
    }
}

```

```

cout << "=== Sorted by area ===\n";

for(int i = 0; i < n; i++){
    cout << shapes[i]->getName() << endl;
    cout << "Talbai: " << shapes[i]->talbai() << endl;
    cout << "Perimetr: " << shapes[i]->perimetr() << endl;
    cout << "-----" << endl;
}

for(int i = 0; i < n; i++){
    delete shapes[i];
}

delete[] shapes;
}

```